Architecture Justification — Student Startup Crowdfunding Platform

Lab 3: Component Modelling & Architectural Pattern Selection — Problem Statement #40

N Bhuvan Teja | PES1UG24CS286 | Software Engineering | 5th Semester — E

Architecture Selection: We chose Layered Architecture for the Student Startup Crowdfunding Platform.

1. Architectural Choice

Layered Architecture with three layers. The Web Client UI is the Presentation layer. Campaign Manager, Milestone Review Service, Payment Service and Notification Service form the Business layer, and one Database Component is the Data layer. The Payment Gateway is an external system reached only through Payment Service. Calls go downward through declared interfaces: ICampaign, IMilestone, IPayment, IGateway, INotify and IDataAccess.

Style	Verdict	Reason for this platform
Layered	Chosen	Single consistent ledger, one deployable unit, clear module boundaries.
Microservices	Rejected	Splits pledge, ledger and approval state across services; heavy operations for a small flow.
Client-Server	Rejected	Centralises everything on the server with no internal boundary between campaign and payment logic.

2. Two Reasons for the Choice

Reason 1: the money state has to stay consistent. A tranche may only be released against a recorded Faculty Committee approval and a funded escrow balance (FR-001, FR-004). A single data layer lets Campaign Manager, Milestone Review Service and Payment Service write pledge, ledger and decision records inside one database transaction. With microservices the same update would span several services and need compensating transactions, and a failure there means a wrong payout.

Reason 2: the scale and team match the pattern. The core flow is five steps (create campaign, pledge, verify milestone, release tranche, notify backers) for a university-scale user base. One application with clean layers is quick to build, test and demonstrate, whereas microservices would add service discovery, inter-service networking and per-service deployment that this flow does not need. Components stay replaceable: the gateway can be swapped by changing what sits behind IGateway.

3. Security Advantage

Card handling is confined to one component. Only Payment Service has a required interface to the gateway, and it sends tokenized authorizations, so no raw card data enters the platform (NFR-001) and the PCI-DSS audit scope shrinks to that component and its gateway link. The UI has no path to the database or the payment code, and every request passes business-layer checks, including the rule that Milestone Review Service calls releaseTranche only after an approval is recorded.

4. Performance Benefit

The business components are deployed together, so the calls from Campaign Manager and Milestone Review Service to Payment Service are in-process calls with no network hop, which is the latency cost counted against microservices. Pledge latency then depends mainly on one external call, the gateway authorization. Notification runs asynchronously over INotify, so a tranche release does not wait for emails to go out, which supports the 5-minute delivery target in FR-005.

Trade-off Accepted

The layers are coupled and the application scales as one unit, so a very large pledge spike would mean replicating the whole application. For the expected load of this platform that costs less than operating separate services.